IIT DELHI

TRACK B

AI powered & Digital In-Vehicle Experience

Raaste Mein

Food stop recommendation from route context and feedback

Design, implementation and prototype evaluation
Installed Android progressive web app, version 5.1.0

Manasjyoti Baishya
Individual entry, IIT Delhi

02 Objective and summary

Overview of the implemented system and the available evidence

Objective

Develop a phone prototype that recommends an upcoming food stop using the route, current time, a spare travel allowance and previous accept/reject choices.

Method

Project locations onto a fixed route. Check availability and travel constraints, rank the remaining venues, issue an advance alert and retain explicit feedback locally.

Evidence and interpretation

The available export contains three completed GPS trips. Screenshots document advance alerts during one evening journey. These support functional demonstration. Recommendation relevance and travel-time accuracy have not been quantified.

Sources: final v5.1.0 code, supplied trip export and eight phone screenshots [S1-S4]

03 Problem and design motivation

Initial user context: students and urban passengers considering food during an existing journey

User situation

Friends looking for a new café must compare places with the route they are already taking.

A busy daily routine can also postpone a meal. A relevant prompt may make a stop easier to consider during that journey.

Engineering problem

A venue may be nearby but already behind the passenger, closed at arrival or outside the available extra-travel time.

The app must choose when to suggest a stop and how to adapt after explicit choices.

Origin of the concept

Swiggy on trains inspired the connection between food and travel. The prototype explores recommending a food stop on a campus route. No delivery service is integrated.

Source: author account, 20 October 2026. These observations are not survey findings [S5].

04

Functional requirements

Requirements define observable prototype behaviour

Requirement	Implemented behaviour
User control	All nine venues remain browseable. No choice is required before starting.
Journey context	GPS supplies position and actual IST time. Spare travel can be set from 0 to 30 min.
Advance timing	Automatic suggestions target an upcoming venue before its route position.
Limited interruption	At most two automatic alerts per trip, with a minimum separation of 250 m.
Explicit adaptation	Accept and Reject alter preferences. Dismissal and timeout remain neutral.
Continuity of data	Local preferences and completed-trip records persist across ordinary app restarts.

The spare-time setting limits estimated extra travel. It does not include food preparation, waiting or eating.

Sources: app.js, suggestion-engine.js, feedback-learning.js and current phone acceptance [S1, S2]

05

Route and input dataset

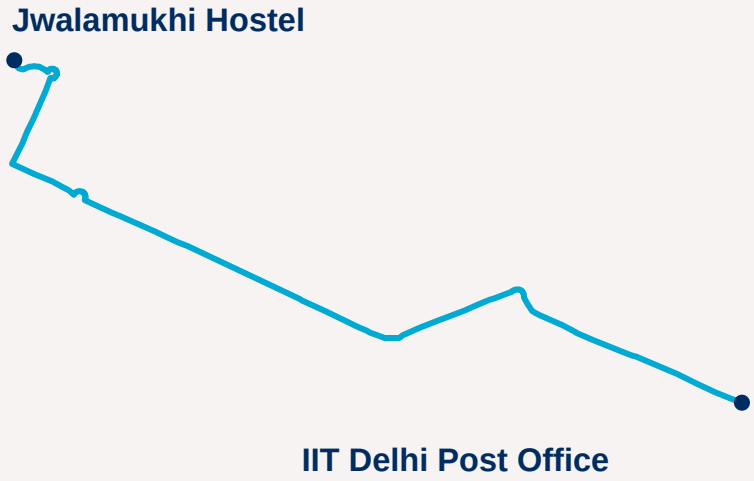
Fixed direction: Jwalamukhi Hostel to IIT Delhi Post Office

Demonstration scope

Route length: 2,048.3 m, shown as 2.05 km.
183 ordered points form 182 line segments.
Nine venues contain 40 menu entries.

Each venue has coordinates, a category, opening hours, a rating and price ranges. These are supplied, saved demonstration data.

The current route has one supported direction. Reverse travel and arbitrary destinations are outside the submitted implementation.



Plan view of the supplied route. North is up.
Uniform scale in a local metric approximation.

Sources: route.json, cafes.json and supplied KML. Geometry computed from the frozen data [S1, S4].

06

Venue data and route projections

Hours and ratings are saved inputs. Distances below derive from route geometry.

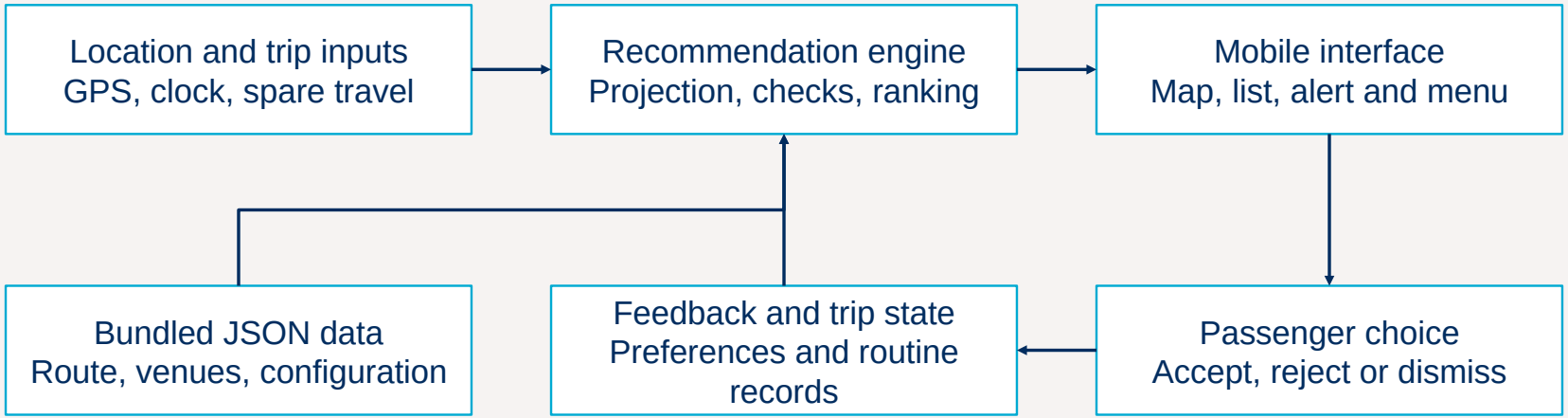
Venue	Scoring category	Saved hours	Rating / 5	Offset (m)	Route position (m)
Rajdhani Chatkare	meals	00:00-24:00	3.7	33	672
Chaayos	tea	10:00-23:00	4.1	26	674
Naivedyam	meals	08:30-22:00	4.2	67	1190
Amul	snacks	08:30-02:00	4.0	76	1163
Nescafé	coffee	08:00-04:00	4.1	67	1169
HPMC	snacks	09:30-20:30	4.0	61	1164
Bikanervala	snacks	10:00-22:00	3.7	21	2048
Tealogy	tea	10:30-23:00	3.8	12	2048
Chicago Pizza	meals	09:00-22:00	3.2	13	2048

Offset is the shortest distance to the route line. Route position is distance along that line from the hostel.
The category is one label per venue, although its menu may contain several types of food or drink.

Sources: cafes.json and measureCafe() in suggestion-engine.js. Values rounded to the nearest metre [S1].

07 System architecture

The recommendation calculations and stored preferences run on the phone



The hosted files provide the app and reference data. Local storage retains feedback and completed-trip metadata. The recommendation engine does not call a cloud model or restaurant service.

Sources: application modules, manifest.json and service-worker.js in v5.1.0 [S1]

08

Live GPS and replay modes

Both modes use the same saved venues and recommendation rules

Aspect	Live GPS	Replay
Position	Trusted phone location projected onto the route	Timer advances a marker along the saved route
Clock	Actual date and IST clock	User-selected departure and replay date
Speed	Measured moving speed, with validation and smoothing	15 km/h simulated travel, played at 1x, 2x or 4x
History	GPS completion requires origin, continuity and arrival checks	Replay completion creates a separate replay record
Purpose	Demonstration of location-triggered operation	Repeatable demonstration of time and preference effects

Until moving speed is available, stop estimates use a labelled 15 km/h assumption. GPS ETA remains unavailable when a reliable moving-speed estimate cannot support it.

Sources: app.js, gps-engine.js and routine-engine.js [S1]

09

Spatial projection and travel estimates

Distances are geometric approximations rather than road-network detours

Location relative to the route

The app projects the phone and each venue onto the closest route segment.
Let s be current route progress, s_i the venue route position and d_i its offset from the route.

$$\text{Extra travel: } \Delta t_i = 2\, d_i / (60\, v)$$

d_i is in metres, v is planning speed in metres per second, and Δt_i is in minutes.
The factor 2 approximates travelling from the route to the venue and returning.

$$\text{GPS arrival clock: } t_i = t_0 + \max(0, s_i - s)/(60\, v) + \Delta t_i/2 + B_i$$

t_0 is the current IST clock and t_i the arrival clock, in minutes. B_i adds accepted travel before venue i.
The arrival estimate excludes queues, food preparation and eating time.

Sources: prepareRoute(), measureCafe(), tripClockMinutes() and scoreCafe() [S1]

10 Availability checks before ranking

An automatic suggestion must satisfy every relevant gate

Check	Condition
Reliable context	In GPS mode, the trip must be active with a fresh, trusted route fix.
Upcoming place	The venue projection must be ahead of current route progress.
Route proximity	Venue offset must be no more than 300 m.
Opening hours	The saved hours must indicate open now and open at estimated arrival.
Travel allowance	Extra travel must fit both the 30 min detour limit and remaining spare travel.
Prompt eligibility	The venue must not have been shown this trip, and score must be at least 0.50.

These checks govern availability and automatic alerts. All nine venues remain in the complete list with their status, even when a venue is unavailable or has a low preference value.

Sources: scoreCafe() and recordVenueChoice() in the final application [S1]

11 Recommendation ranking

A fixed weighted score orders venues that pass the availability checks

$$S_i = 0.60\,A_i + 0.20\,T_i + 0.10\,R_i + 0.10\,D_i$$

Term	Definition	Weight
A_i: preference	0.8 x venue preference + 0.2 x category preference	0.60
T_i: time fit	Saved category value for the predicted arrival time band	0.20
R_i: rating	Saved venue rating divided by 5	0.10
D_i: detour fit	1 - $\Delta t_i/30$, restricted to the interval from 0 to 1	0.10

Every component lies between 0 and 1. The weights are chosen configuration values.
They have not been fitted to a training dataset or validated as an optimum.

Sources: scoreCafe() and weights in demo-settings.json [S1]

12

Time-of-day context model

The current time-fit values are developer-defined assumptions

Arrival time band	Tea	Coffee	Snacks	Meals
morning 06:00-12:00	1.00	1.00	0.70	0.40
lunchtime 12:00-15:00	0.55	0.60	0.70	1.00
afternoon 15:00-17:00	0.85	0.80	0.85	0.50
evening 17:00-20:00	1.00	0.80	1.00	0.60
night 20:00-06:00	0.65	0.50	0.70	0.90

A value of 1 gives the largest time-fit contribution in the model. It is not a measured probability of wanting that category. Preference, rating and travel still affect the score.

Source: timeFits in data/demo-settings.json. Categories refer to each venue label [S1].

13

Automatic alert scheduling

The number and timing of prompts differ from the number of available venues

Policy	Implementation
GPS approach window	Consider the highest-scoring eligible upcoming candidate. Show it when 300-400 m ahead.
Replay trigger	Use the configured point 350 m before the venue route position.
Trip interruption limit	Show no more than two automatic suggestions during one trip.
Separation	Require at least 250 m of route progress between shown suggestions.
Dismissal	Close after 12 s, or by close/outside tap/swipe. This does not record a rejection.
Competing screens	Suspend new alerts while a menu or settings/history panel is open.

A venue can fit the travel allowance and still receive no alert. Ranking, spacing, the two-alert cap and a missed approach window can all affect which venues appear.

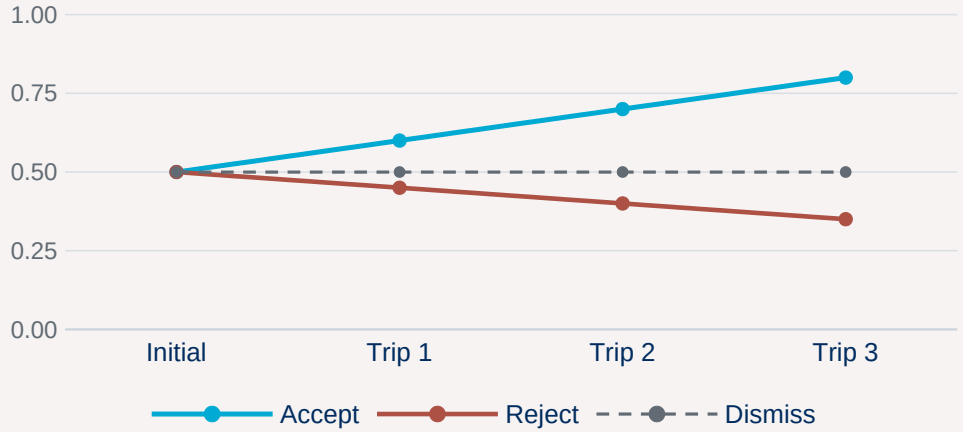
Sources: findLiveSuggestion(), findSuggestionBetween() and showSuggestion() [S1]

14 Preference update from explicit feedback

The learning mechanism is a bounded additive update with persistent local state

New preference = min(1, max(0, previous preference + feedback step))

Illustration: feedback on the same venue



Update rules

Accept adds 0.10.
Reject subtracts 0.05.
Dismissal makes no change.

The venue and its category both update.
One final venue choice counts per trip.
Later trips use the revised preferences.

Sources: feedback-learning.js and configured update steps. Plot is illustrative, not observed trips [S1].

15

Explanation and routine rules

The app uses explicit conditions before making a statement about user behaviour

Output	Condition or basis
Previous-choice remark	State the saved count when a venue has earlier accepts or rejects.
Usual-choice remark	At least three accepts for the venue and an acceptance share of at least two-thirds.
Time-specific usual choice	Also require at least three accepts in that time band, exceeding its rejects.
Meal or drink context	Use the time band and saved menu/category, for example a lunchtime noodle option.
Routine match	At least three completed trips of one source, same weekday/weekend group, endpoints within 150 m and a total departure span no greater than 30 min.

GPS, replay and sample histories stay separate. The three available GPS departures do not satisfy the 30 min grouping condition, so this export supports no detected routine.

Sources: smartRemark(), detectRoutines() and matchingRoutine(). Export has no detected routine [S1, S3].

16 Menus and simulated pre-orders

The menu flow demonstrates a possible interaction beyond the recommendation

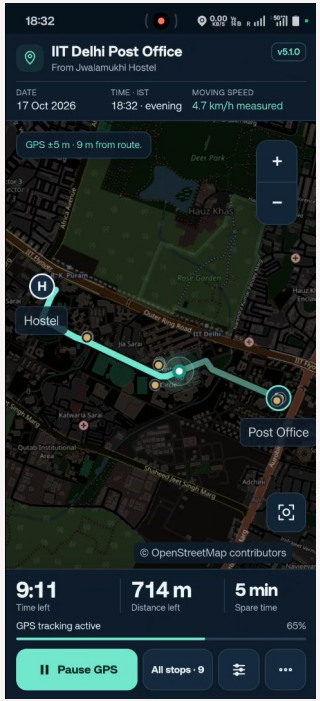
Function	Current implementation
Menu catalogue	Show supplied price ranges for all 40 saved menu entries.
Cart	Allow integer quantities from 0 to 10 per item.
Demo amount	Use each range lower endpoint as the labelled demonstration unit price.
Confirmation	Require an available venue and a non-empty cart. Confirmation records one Accept choice.
Receipt	Create an in-session demo receipt with items, amount and the estimated arrival at confirmation.

No order reaches a restaurant. Payment, stock, queue length and preparation time are absent.
The displayed ready-by value is an arrival estimate, not a restaurant commitment.

Sources: order-engine.js, app.js and cafes.json [S1]

17 Final mobile interface

The installed phone app is the demonstrated prototype interface



Information during an active journey

A compact header displays date, IST time and measured speed.
The map occupies approximately 65% of the phone display.
A moving marker shows progress on the saved route.

Bottom controls show remaining distance, ETA and spare travel.
Trip settings minimise after the journey starts.
The complete venue list and tools remain accessible.

Interaction behaviour

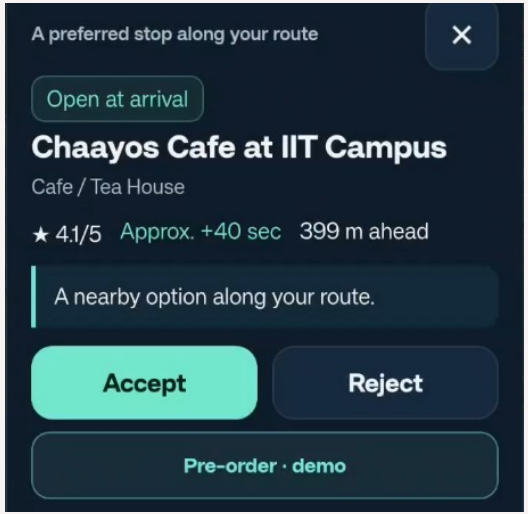
GPS continues while menus and history panels are open.
New automatic alerts wait until those panels close.

Source: screenshot 4, 17 October 2026 at 18:32 IST. Map © OpenStreetMap contributors [S2].

18 Observed advance suggestions

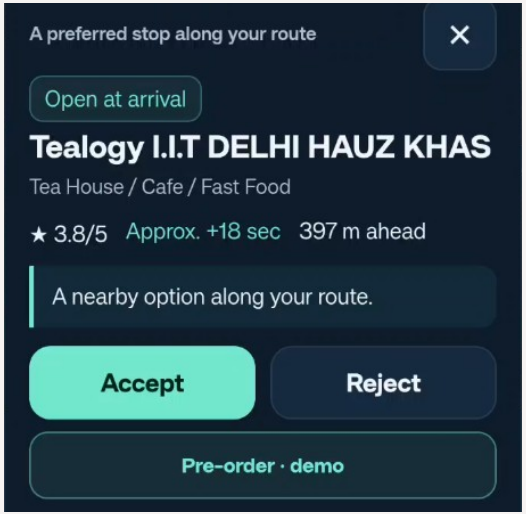
Two recorded alerts from the same evening GPS journey

Chaayos, 18:19 IST



399 m ahead, approximately +40 s travel.
The passenger can accept, reject or dismiss.

Tealogy, 18:36 IST



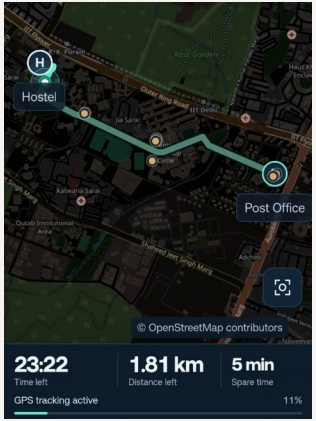
397 m ahead, approximately +18 s travel.
The card indicates open at predicted arrival.

Sources: screenshots 2 and 6, 17 October 2026. Values are on-screen estimates [S2].

19 Observed progression on the route

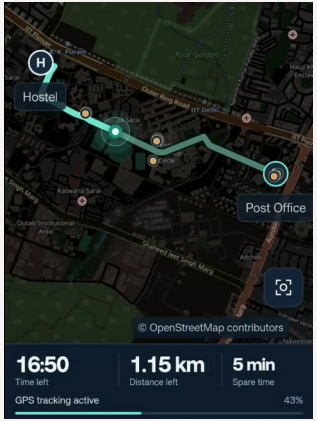
Selected frames from one evening journey, 17 October 2026

18:18 IST



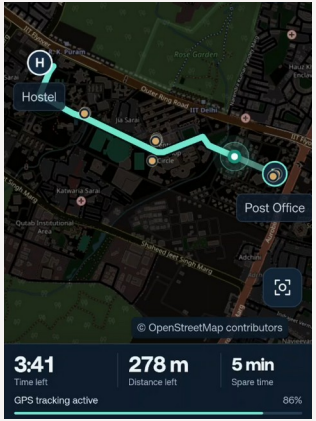
1.81 km remaining
11% progress

18:26 IST



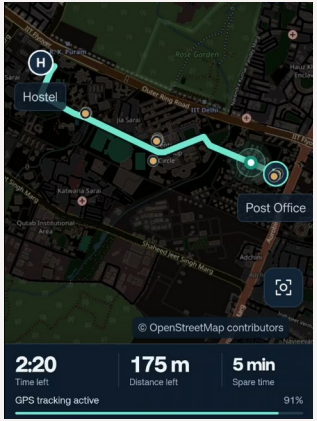
1.15 km remaining
43% progress

18:38 IST



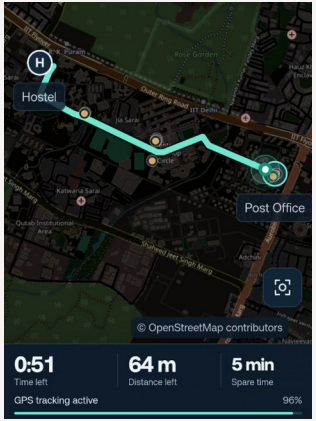
278 m remaining
86% progress

18:39 IST



175 m remaining
91% progress

18:40 IST



64 m remaining
96% progress

These frames show changing route progress. The last view is near arrival, with 64 m still remaining.

Sources: screenshots 1, 3, 5, 7 and 8. Maps © OpenStreetMap contributors [S2].

Evidence from the saved trip export

The available export provides completed-trip metadata and explicit choice events

Record	Date	Departure IST	Source	Completed
gps-1	2026-10-17	12:09	gps	yes
gps-2	2026-10-17	12:49	gps	yes
gps-3	2026-10-17	18:16	gps	yes

Retained feedback in this export

One explicit event records an Accept for Naivedyam at lunchtime.
Reconstructed venue preference is 0.60. The meals-category preference is also 0.60.
No Reject event and no completed replay record appear in this particular export.

The profile also contains 45 session entries, many without feedback. Starts and resets create such entries.
They must not be counted as 45 completed journeys or 45 independent learning observations.

Source: InVehicle_History_2026-10-17 (3).json, supplied by the author [S3]

21 Data persistence and backup

The phone stores app history locally, scoped to its web origin

Data	Storage and lifecycle
Choice profile	localStorage retains trip sessions and explicit venue/category choice events.
Completed-trip history	A separate localStorage record retains source, date, departure and endpoints.
Current journey state	Progress, planned stops, cart and demo receipts are transient.
Backup	JSON export includes the choice profile and completed-trip history for this route.
Restore	Validate format, route and records, preview counts, then explicitly replace local history.

The app stores no continuous GPS trajectory in the backup and uses no server database.
A different host or device needs a backup transfer. Ordinary trip reset keeps the saved history.

Sources: persistLearning(), persistHistory() and history-backup.js [S1]

22 Hosting, installation and offline behaviour

A progressive web app is a website installed and launched on the phone as an app

Component	Current implementation
Delivery	Static HTML, CSS, JavaScript and JSON hosted over HTTPS on chatgpt.site.
Installed interface	Standalone portrait mode, app icons, start URL and scope defined in manifest.json.
Offline app files	A versioned service worker caches the bundled app, route, menus and helper modules.
Maps	OpenStreetMap tiles remain network resources. The route has a plain-background fallback.
Updates	Keep one cached build together. The user activates an available app update.

Cached replay and menus can run without a connection. A usable live location still depends on the phone location service. Offline or airplane-mode GPS is not guaranteed.

Sources: manifest.json, service-worker.js, pwa.js and final deployment configuration [S1]

23

Scope of the available evidence

Functional demonstration does not establish recommendation performance

Question	Available support	Current limit
Does the phone workflow run?	Installed-app screenshots and author acceptance	One author and one campus route
Do alerts precede a stop?	Chaayos and Tealogy frames show 399 m and 397 m ahead	No complete alert log or detection-rate denominator
Can history affect ranking?	Explicit event, deterministic update and retained preference	No controlled relevance comparison
Is the detour estimate accurate?	Geometric travel model is implemented	No measured side-trip times for error analysis
Are routine and offline features evaluated?	Rules and cache behaviour exist in source, plus user checks	No matched routine exists in the supplied export

No claim is made for time saved, fewer missed meals, ranking accuracy or production readiness.
Weights and thresholds are prototype choices. High-speed and in-vehicle integration remain unvalidated.

Sources: supplied screenshots and export, code review and author-confirmed demonstrations [S1-S5]

24

Feasibility and conclusions

The submission demonstrates a working recommendation loop on existing phone hardware

Implemented contribution

The prototype combines route-based advance timing with travel constraints, a transparent ranking rule and persistent feedback. It retains the passenger’s ability to browse and decline.

Feasibility within the demonstrated scope

The app uses existing phone GPS and local storage. It needs no additional recommendation server or bespoke hardware. The available evidence supports operation on the fixed campus route.

Proposed application for Hyundai

A future Hyundai passenger display could use vehicle location and trip context for this service. That would require integration and validation. Verified venue data, road-network detours and waiting/eating duration remain possible extensions.

Sources: final implementation, recorded demonstration and author project scope [S1-S5]

Appendix A: configuration defaults

Additional parameters retained in data/demo-settings.json

Parameter group	Default or rule
Spare travel and prompt score	Default spare travel 5 min, allowed 0-30 min. Prompt score threshold 0.50.
Venue and alert geometry	Venue offset <=300 m. GPS alert window 300-400 m. Prompt separation >=250 m.
GPS quality	Reported accuracy <=50 m. Fix age <=15 s. Future timestamp tolerance 5 s.
Jump rejection	Displacement allowance = elapsed seconds x 12 m/s + 30 m.
Arrival	Within 30 m of destination and within 50 m along route, with two fixes >=1 s apart.
Completed GPS record	Start within 75 m. No interruption gap >30 s or forward step >250 m. Accumulated valid forward progress >=80% of route.
Routine and preference	Routine: >=3 matching trips, <=30 min departure span, endpoints <=150 m. Preference starts 0.50, Accept +0.10, Reject -0.05.

Source: GPS, routine and suggestion configuration in the frozen v5.1.0 files [S1]

Appendix B: worked score calculation

Illustrative fixed inputs, chosen to explain the implemented equations

Assume a venue offset of 30 m, planning speed of 15 km/h, rating 4.1/5 and time fit 1.00. Both venue and category preference begin at 0.50. Assume all availability checks pass.

Quantity	Calculation	Value
Extra travel	2 x 30 / (60 x 15/3.6)	0.24 min = 14.4 s
Detour fit D	1 - 0.24 / 30	0.992
Rating term R	4.1 / 5	0.820
Preference A	0.8 x 0.50 + 0.2 x 0.50	0.500
Initial score S	0.60 x 0.50 + 0.20 x 1 + 0.10 x 0.82 + 0.10 x 0.992	0.6812
After three accepts	Both preferences become 0.80, with other inputs fixed	Score 0.8612

Source: scoreCafe() and configured weights. This example is not a field measurement [S1].

Appendix C: reproduction procedure

The frozen source package contains the final installed-phone prototype

App and reference data

1. Extract the v5.1.0 package and serve its app files from a static HTTPS root.
2. Retain the relative paths, JSON files, vendored Leaflet assets, manifest and service worker.
3. Open the address online, allow the complete offline cache, then install on Android.
4. Start Replay with 11:00 IST and 5 min spare travel for the configured baseline.
5. For live operation, allow phone location and start near the hostel on the supported route.

Preference and history state

A new origin with no saved state starts from neutral preferences. To reproduce the recorded state, restore the supplied history JSON through the backup interface after checking the route. Ordinary app updates on the same origin retain the stored history.

Source package: InVehicle_v5_1_0_MobileMap.zip. Personal history is a separate backup [S1, S3].

Appendix D: source and evidence register

Claims in the presentation refer to the following project materials

Reference	Material	Role in this presentation
S1	Frozen app source and JSON configuration	Implementation, algorithms, thresholds, menus, storage, PWA and reproduction
S2	Eight supplied phone screenshots, 17 October 2026	Installed layout, Chaayos/Tealogy alerts and selected progress frames
S3	InVehicle_History_2026-10-17 (3).json	Three GPS completion records and one explicit retained choice event
S4	Simulated route inside IITD.kml and route.json	Route geometry and fixed demonstration endpoints
S5	Author account and final prototype acceptance	Motivation, intended passenger scenario and confirmation of demonstrations

Core modules: app.js, suggestion-engine.js, gps-engine.js, feedback-learning.js, routine-engine.js, order-engine.js and history-backup.js. Delivery files include manifest.json, pwa.js and service-worker.js. Leaflet 1.9.4 and OpenStreetMap provide map rendering and tiles.

All code, configuration and screenshot references concern the final v5.1.0 prototype.